Data Structures & Algorithms by devHafsa

This course will get you prepared for placements and teach you how to create efficient and fast algorithms.

> **Data structures and algorithms are two different things:**
>
> - Data Structures: Arrangement of data in memory (data items) so that they can be used efficiently.
> - Algorithms: Sequence of steps to solve a given problem using efficient data structures.

Other Terminology

- **Database:** Collection of information in permanent storage for faster retrieval and updation.
- **Data Warehousing:** Management of huge amount of legacy data for better analysis.
- **Big Data:** Analysis of too large or complex data which cannot be dealt with traditional data processing applications.

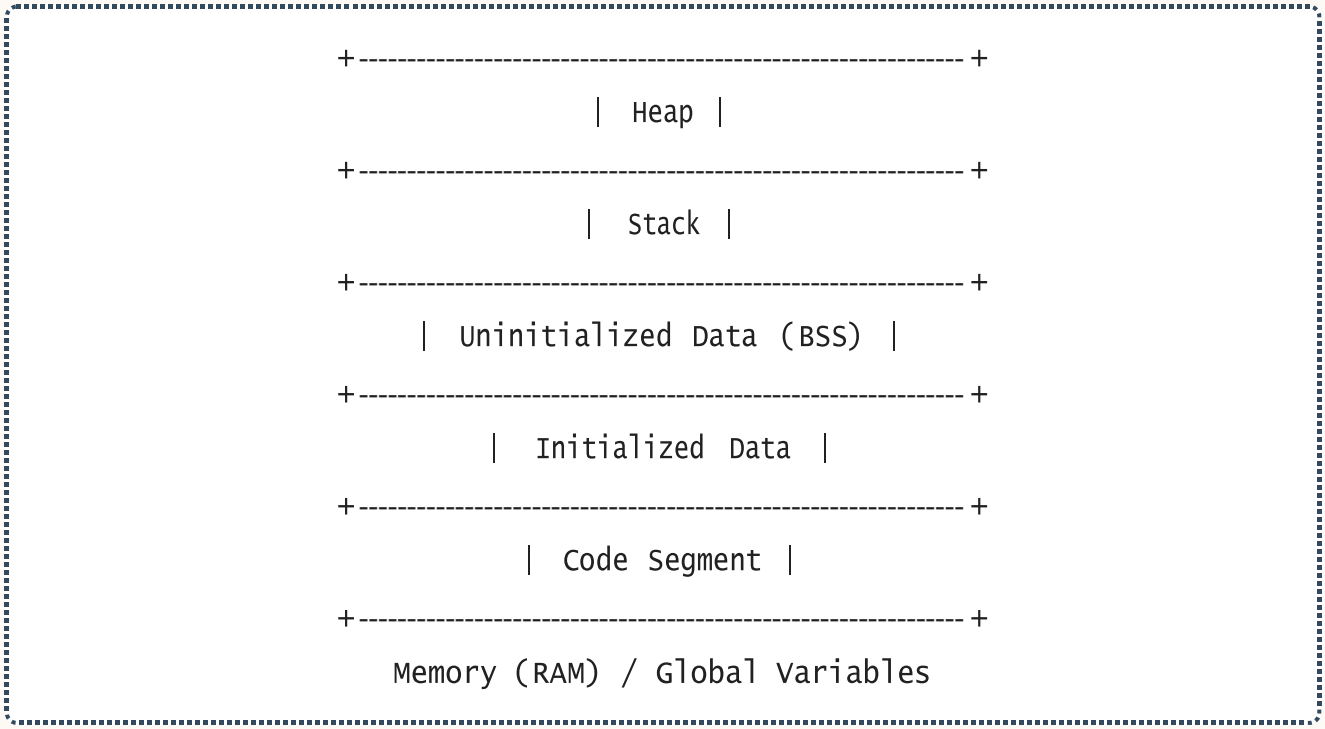
Data Structures and Algorithms are nothing new. If you have done programming in any language like C, you must have used `Arrays` and sequence of processing steps to solve a problem (Algorithm).

```
[Problem] → [Algorithm] → [Data Structures in Memory] → [Solution]
```

Memory Layout of C Programs

When the program starts, its code is copied to the main memory.

Stack holds the memory occupied by the functions.



Heap contains the data which is requested by the program as dynamic memory.

Initialized and uninitialized data segments hold initialized and uninitialized global variables respectively.

Time Complexity & Big O Notation

This morning I wanted to eat some pizzas; so I asked my brother to get me some from Domino's (3km far). He got me the pizza and I was happy only to realize it was too less for 29 friends who came to my house for a surprise visit!

My brother can get 2 pizzas for me on his bike, but pizza for 29 friends is too huge of an input for him which he cannot handle!

2 pizzas → Okay! Not a big deal!

68 pizzas→Not possible!!

> **What is Time Complexity?**
>
> Time Complexity is the study of efficiency of algorithms. **Time Complexity =** How time taken to execute an algorithm grows with the size of the input.

Consider two developers who created an algorithm to sort numbers. Shubham and Rohan did this independently.

When ran for input size *n*, following results were recorded:

No of elements (*n*)	Shubham's Algo	Rohan's Algo
10 elements	90 ms	122 ms
70 elements	110 ms	124 ms
110 elements	180 ms	131 ms
1000 elements	2s	800 ms

We can see that initially Shubham's algorithm was shining for smaller input but as the number of elements increases, Rohan's algorithm looks good!

Quick Quiz: Whose algorithm is better?

Time Complexity: Sending GTA V

Let us say you have a friend living 5 km away from your place. Final exams are over and you want him to get this 60 GB file (GTA V game) from you. How will you send it to him? Note that both of you are using 4G with 1 GB/day data limit.

The best way to send him the game is by delivering it to his house. Copy the game to a Hard disk and send it!

Will you do the same thing for sending a game like minesweeper which is in KBs of size? No, because you can send it via internet.

- As the file size grows, time taken by online sending increases linearly $O(n)$.
- As the file size grows, time taken by physical sending remains constant $O(1)$.

Calculating Order in terms of Input Size

In order to calculate the order, most impactful term containing *n* is taken into account.

Let us assume that formula of an algorithm in terms of input size *n* looks like this:

Algo 1 $\rightarrow k_1 n^2 + k_2 n + 36 \Rightarrow$ $O(n^2)$ (Highest order term)

Algo 2 $\rightarrow k_1 k_2{}^2 + k_3 k_2 + 8 \Rightarrow$ $O(n)$

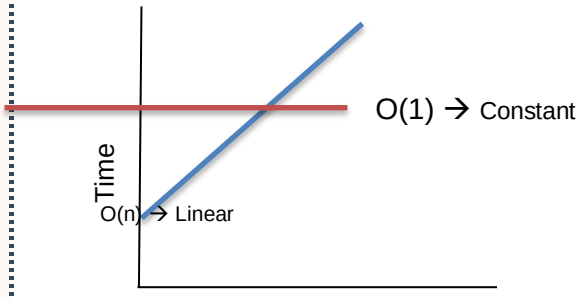
or

Algo 2 $\rightarrow k_1 k_2{}^2 n^0 + k_3 k_2 + 8 \Rightarrow$ $O(1)$ **Or** $O(n^0)$

Note that these are the formulas for time taken by them.

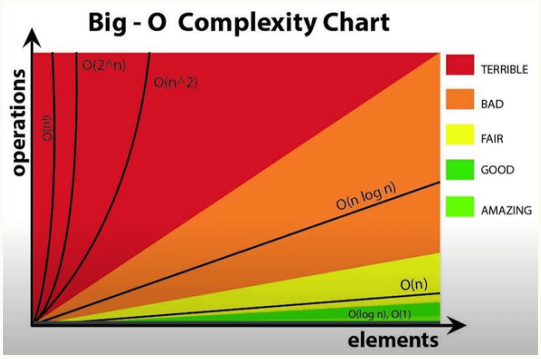
Visualising Big O

We were to plot O(1) and O(n) on a graph, they will look something like this:



n

Big-O Complexity Chart



Source: https://stackoverflow.com/questions/3255/big-o-how-do-you-calculate-approximate-it

Asymptotic Notations

Asymptotic notations give us an idea about how good a given algorithm is compared to some other algorithm. Let us see the mathematical definition of "order of" now.

Primarily there are three types of widely used asymptotic notations:

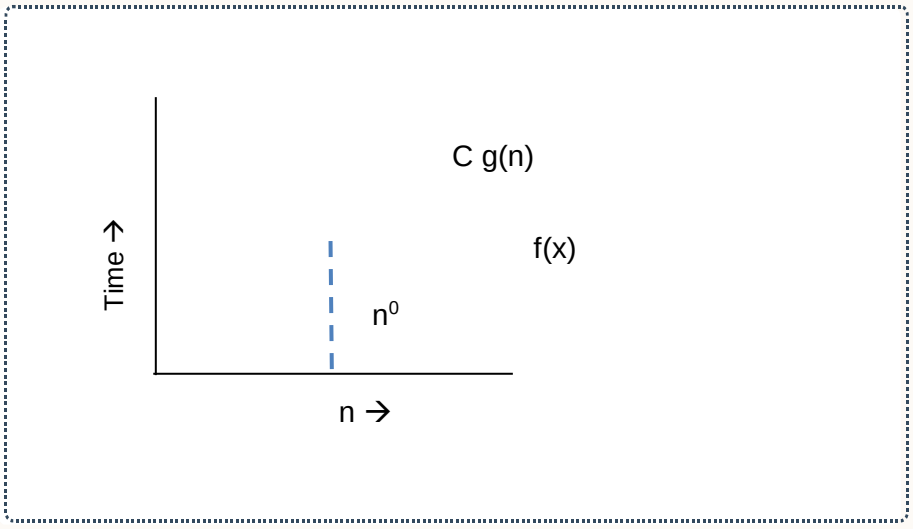
1. Big Oh notation (O)
2. Big Omega notation (Ω)
3. Big Theta notation (Θ) → Widely used one!

Big Oh Notation

> Big Oh notation is used to describe asymptotic **upper bound**. Mathematically, if f(n) describes running time of an algorithm; f(n) is O(g(n)) iff there exist positive constants *c* and n_0 such that:
>
> $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

If a function is O(n), it is automatically $O(n^2)$ as well.

Graphic Example for Big Oh (O)

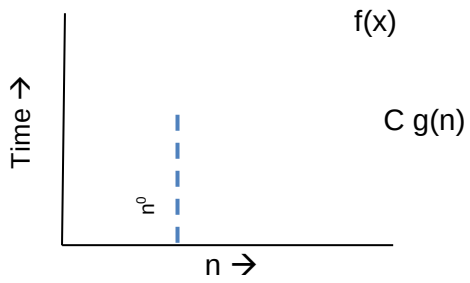


Big Omega Notation (Ω)

> Just like O notation provides an asymptotic upper bound, Ω notation provides asymptotic **lower bound**. Let f(n) define running time of an algorithm; f(n) is said to be Ω(g(n)) if there exists positive constants *c* and n_0 such that:
>
> $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$

If a function is $O(n^2)$, it is automatically O(n) as well.

Graphic Example for Big Omega (Ω)

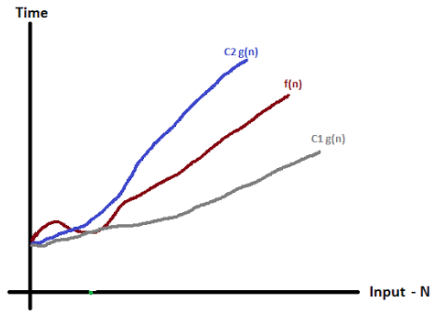


Big Theta Notation (Θ)

> Let f(n) define running time of an algorithm. f(n) is said to be Θ(g(n)) iff f(n) is O(g(n)) **and** f(n) is Ω(g(n)).
>
> Mathematically:
>
> $0 \leq f(n) \leq c_1 \cdot g(n)$ ∀ $n \geq n_0$ (sufficiently large value of n)
>
> $0 \leq c_2 \cdot g(n) \leq f(n)$ ∀ $n \geq n_0$
>
> Merging both equations: $\mathbf{0 \leq c_2 g(n) \leq f(n) \leq c_1 g(n)}$ ∀ $n \geq n_0$

The equation simply means these sandwiched constants $c_2 g(n)$ and $c_1 g(n)$ bound f(n) tightly from above and below.

Graphic Example of Big Theta (Θ)



Which one of these to use? Since Big Theta gives a better picture of runtime for a given algorithm, most of the interviewers expect you to provide an answer in terms of Big Theta (Θ) of *n*.

Quick Quiz: Prove that $n^2 + n + 1$ is $O(n^3)$, $O(n^2)$ and $\Theta(n^2)$ using respective definitions.

Increasing Order of Common Runtimes

$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n^n$

Common runtimes from better to worse: Better ← [O(1) → O(log n) → O(n) → O(n log n) → $O(n^2)$] → Worse

Best, Worst and Expected Case

Sometimes we get lucky in life: Exams cancelled when you were not prepared, surprise test when you were prepared, etc. →Best Case

Sometimes we get unlucky: Questions you never prepared asked in exams, rain during sports period, etc. →Worst Case

Overall the life remains balanced with the mixture.

Analysis of a Search Algorithm

Consider an array which is sorted in increasing order: `[1, 7, 18, 28, 50, 180]`. We have to search a given number in this array and report whether it's present in the array or not.

- **Algo 1 :** Start from first element until an element greater than or equal to the number to be searched is found.
- **Algo 2:** Check whether the first or the last element is equal to the number. If not, find the number between these two elements (center of the array). If the center element is greater than the number to be searched, repeat the process for first half else repeat for second half until the number is found.

Analyzing Algo 1

If we really get lucky, the first element of the array might turn out to be the element we are searching for. Hence we made just one comparison.

Best Case Complexity = O(1)

If we are really unlucky, the element we are searching for might be the last one.

Worst Case Complexity = O(n)

For calculating Average Case time, we sum the list of all possible cases' runtime and divide it with the total number of cases. (Sometimes calculation of average case time gets very complicated).

Analyzing Algo 2

If we get really lucky, the first element will be the only one which gets compared.

Best Case Complexity = O(1)

If we get unlucky, we will have to keep dividing the array into halves until we get a single element (the array gets finished).

Worst Case Complexity = O(log n)

What is log(n)? What is that?

log(n) = Number of times you need to half the array of size *n* before it gets exhausted.

```
log 8 = 3 → 8/2 → 4/2 → 2/2 → 1  (can't break anymore: 1+1+1 = 3)
      log 4 = 2 → 4/2 → 2/2 → 1  (can't break anymore 1+1 = 2)
```

log n simply means how many times I need to divide units such that we cannot divide them (into halves) anymore.

Space Complexity

Time is not the only thing we worry about while analyzing algorithms. Space is equally important.

- Creating an array of size *n* → O(n) Space.
- If a function calls itself recursively *n* times, its space complexity is O(n).

Quick Quiz: Calculate space complexity of a function which calculates factorial of a given number *n*.

Why can't we calculate complexity in seconds?

Not everyone's computer is equally powerful! Asymptotic analysis is the measure of how time (runtime) grows with input.

Techniques to Calculate Time Complexity

Once we are able to write the runtime in terms of size of the input (*n*), we can find the time complexity.

For example: $T(n) \rightarrow n^2 \rightarrow O(n^2)$, $T(n) \rightarrow \log n \rightarrow O(\log n)$

Some Tricks to Calculate Complexity:

1. **Drop the Constants:** Anything you might think is O(3n) is O(n).
2. **Drop the Non-Dominant Terms:** Anything you represent as $O(n^2 + n)$ can be written as $O(n^2)$.
3. **Consider all variables which are provided as input:** $O(m \cdot n)$ might exist for some cases! In most of the cases, we try to represent the runtime in terms of the input which can be more than one in number. For example: Painting a park of dimension $m \times n \rightarrow O(m \cdot n)$.

Time Complexity - Competitive Practice Sheet

1 Find the time complexity of the `func1` function in the program shown in `program1.c` as follows:

```
#include <stdio.h>
void func1(int array[], int length)
{
    int sum = 0;
    int product = 1;
    for (int i=0; i < length; i++)
    {
        sum += array[i];
    }
    for (int i=0; i < length; i++)
    {
        product *= array[i];
    }
}
int main()
{
    int arr[] = {3, 5, 66};
    func1(arr, 3);
    return 0;
}
```

2 Find the time complexity of the `func` function in `program2.c`:

```
void func(int n)
{
    int sum = 0;
    int product = 1;
    for (int i=0; i < n; i++)
    {
        for (int j=0; j < n; j++)
        {
            printf("%d %d\n", i, j);
        }
    }
}
```

3 Consider the recursive algorithm below, where the `random(int n)` spends one unit of time to return a random integer which is evenly distributed within the range `[0, n]`. If the average processing time is T(n), what is the value of T(6)?

```
int function(int n)
{
    int i;
    if (n <= 0)
    {
        return 0;
    }
    else
    {
        i = random(n-1);
        printf("this\n");
        return function(i) + function(n-1 - i);
    }
}
```

4 Which of the following are equivalent to O(N)? Why?

a) O(N + P), where P < N/9

b) O(9N - k)

c) O(N + 8 log N)

d) $O(N + M^2)$

5 The following simple code sums the values of all the nodes in a balanced binary search tree. What is its runtime?

```
int sum(Node node)
{
    if (node == NULL)
    {
        return 0;
    }
    return sum(node.left) + node.value + sum(node.right);
}
```

6 Find the complexity of the following code which tests whether a given number is prime or not?

```
int isPrime(int n) {
    if (n == 1) {
        return 0;
    }
    for (int i = 2; i * i < n; i++) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
```

7. What is the time complexity of the following snippet of code?

```
int isPrime(int n) {
    for (int i = 2; i * i < 10000; i++) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
isPrime();
```

Operations on an Array

Following operations are supported by an array: Traversal, Insertion, Deletion, Search, Sorting (asc, desc), etc.

Traversal

Visiting every element of an array once → Traversal.

Why traversal? For use cases like:

- Storing all elements using `scanf`
- Printing all elements using `printf`

> **An important note about arrays:** In C language, we create an array of length 100 using `a[100]`. We may not use all the elements. It is possible for a program to use just 60 elements out of these 100. But we cannot go beyond 100 elements.

An array can easily be traversed using a for loop in C language.

Insertion

An element can be inserted in an array at a specified position. In order for this operation to be successful, the array should have enough capacity.

```
[7] [10] [11] [13] [ ] → Insert 12 at index 2 → Elements need to be
                shifted to maintain relative order.
```

When no position is specified, it's best to insert the element at the end.

Deletion

An element at specified position can be deleted creating a void which needs to be fixed by shifting all the elements to the left as follows:

```
[7] [10] [11] [13] [8] → Delete 11 at ind 2 → [7] [10] [13] [8] [ ]
                        (Shift elements left)
```

We can also bring the last element of the array to fill the void if the relative ordering is not important.

Searching

Searching can be done by traversing the array until the element to be searched is found.

For sorted array, time taken to search is much less than unsorted array!!

Sorting

Sorting means arranging an array in order (asc or desc). We will see various sorting techniques later in the course.

```
Unsorted array: [12] [7] [18] [1] [8]
 Sorted array: [1] [7] [8] [12] [18]
```

Linear Vs Binary Search

Linear Search

Searches for an element by visiting all elements sequentially until the element is found. For ex, [7,10,2,9,11,21,3] → Can be sorted or unsorted

Search 2, Element 2 is found at index 2, **Worst Case Complexity:** O(n)

Binary Search

Searches for an element by breaking the search space into half in a sorted array.

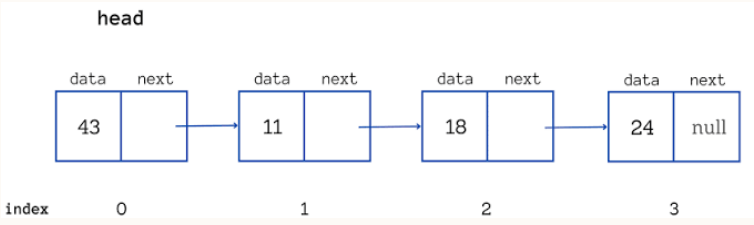
Worst Case Complexity: O(log n)

The search continues towards either side of mid based on whether the element to be searched is lesser or greater than mid.

Linear Search	Binary Search
Works on both sorted and unsorted arrays	Works only on sorted arrays
Equality operations	Inequality operations
O(n) WC Complexity	O(log n) WC Complexity

Introduction to Linked Lists

Linked list are similar to arrays… In Linked Lists, elements are stored in non-contiguous memory locations:



Why Linked Lists?

Memory and the capacity of an array remains fixed. In case of linked lists, we can keep adding and removing elements without any capacity constraints.

Drawbacks of Linked Lists

- Extra memory space for pointers is required (for every node 1 pointer is needed).
- Random access not allowed as elements are not stored in contiguous memory locations.

Implementation

Linked list can be implemented using a structure in C language (Self-referencing structure):

```
struct Node
{
    int data;
    struct Node *next;
};
```

Deletion in a Linked List

Consider the following Linked List: `head → [8] → [9] → [11] → [7] → [2] → NULL`

Deletion can be done for the following cases:

1. Deleting the first Node
2. Deleting the node at an index
3. Deleting the last Node
4. Deleting the first node with a given value

The deletion just like insertion is done by rewiring the pointer connections, the only caveat being: we need to free the memory of the deleted node using `free()`.

Case 1: Deleting the first node

```
struct Node *p = head;
head = head->next;
free(p);
```

Case 2: Deleting the node at an index

For deleting a given node, we first bring a temporary pointer `p` before element to be deleted and `q` on the element being deleted.

```
p->next = q->next;
free(q);
```

Case 3: Deleting the last Node

Can be deleted just like Case 2 by bringing `p` on second last element and `q` on last element.

```
p->next = NULL;
free(q); // To free the memory!
```

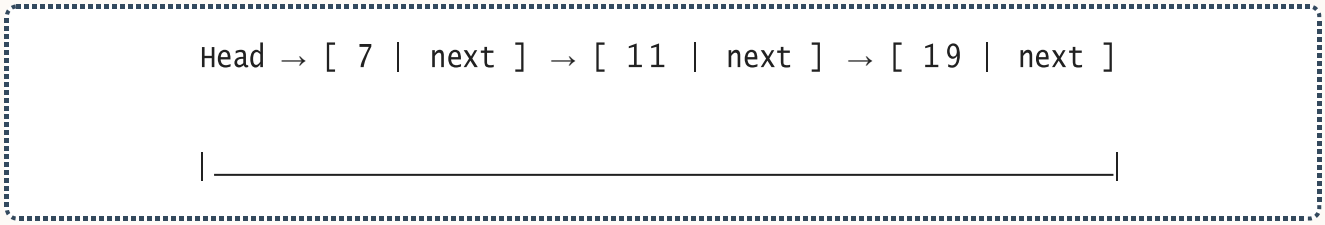
Case 4: Delete the first node with a given value

This can be done exactly like Case 2 by bringing pointers `p` & `q` to appropriate positions.

```
p->next = q->next;
free(q);
```

Circular Linked List

> A circular linked list is a linked list where the last element points to the first element (head) hence forming a circular chain.



Operations on a Circular Linked List

Operations on a circular linked list can be performed exactly like a singly linked list.

Doubly Linked List

> In a doubly linked list, each node contains a data part along with two addresses, one for the previous node and the other one for the next node.

```
NULL ← [prev | 7 | next] <-> [prev | 18 | next] → NULL
```

Implementation

A doubly linked list can be implemented in C language as follows:

```
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};
```

Operations on a Doubly Linked List

The insertion and deletion on a doubly linked list can be performed by rewiring pointer connections just like we saw in a singly linked list. The difference here lies in the fact that we need to adjust two pointers (prev & next) instead of one (next).

Introduction to Stack Data Structure

> Stack is a linear data structure. Operations on stack are performed in **LIFO (Last In First Out)** order. Insertion/deletion can happen on only one end (top).

Item 1 which entered the basket last will be the first one to come out.

Applications of Stack

- Used in function calls.
- Infix to postfix conversion (and other similar conversions).
- Parenthesis matching & more...

Stack ADT

In order to create a stack we need a pointer to the topmost element along with other elements which are stored inside the stack.

Some of the operations of stack ADT are:

1. `push()` → push an element into the stack
2. `pop()` → remove the topmost element from the stack
3. `peek(index)` → value at a given position is returned
4. `isEmpty() / isFull()` → determine whether the stack is empty or full

Implementation of Stack

A stack is a collection of elements with certain discipline. Stack can be implemented using an array or a linked list.

